PennyPilot

Project One – Final Project Report

Mobile App Development | Fall 2026

Akshitha Sainath Sanagarapu and Mouni Sri Nallapaneni

1. Introduction

For our first mobile application development project, we built PennyPilot, an offline finance app focused on the everyday needs of students. We chose this topic because it is easy to lose track of small expenses over a month. A few grocery trips, transportation costs, and food purchases can add up before we notice. We wanted to create something that makes those expenses easier to see, while also giving users a way to set savings goals.

We did not want PennyPilot to be only an expense-entry form. Our idea was to connect transaction history, monthly budgets, savings challenges, and financial insights so that users could understand their overall situation from one application. We also wanted the main features to work without an account or internet connection. That decision influenced how we designed the storage and the rest of the app.

2. What we built

2.1 Dashboard and monthly budget

The Home screen brings together the user's budget, amount spent, amount remaining, progress indicator, and recent activity. We added spending-status messages so the user can notice when they are approaching their monthly limit. In our demo data, for example, a $500 budget with $411 in spending shows 82% used and $89 remaining. The warning makes that number easier to interpret instead of leaving the user to calculate it.

2.2 Transactions and history

Users can record income and expenses, select categories, and add descriptions. We worked on input validation because an incomplete or incorrect entry would affect the rest of the app. We also added editing and deletion, and improved the History screen with search, income/expense filters, and sorting by date or amount. We included CSV export and native sharing so users can keep a copy of their records outside the app.

2.3 Savings challenges

The savings section lets users create a goal, choose a target amount, and record contributions. We expanded this area with challenge validation, editing, filtering, sorting, and a history of progress. Saving a contribution needs to update both the stored contribution information and the progress shown to the user. That relationship became an important part of our database and state-management work.

2.4 Financial insights and settings

The Insights screen summarizes spending by category and presents local financial feedback. We worked on spending anomalies, time-aware analysis, and forecast-related history. When there are no completed forecasts to compare with actual expenses, the app displays an empty-state message rather than claiming an accuracy figure. We also added a persistent theme preference so a user's appearance choice is restored after reopening the app.

3. Technology and design decisions

We used Flutter and Dart for the interface and application logic. We used Provider/ChangeNotifier to share financial state across screens. This was useful because a transaction update can affect the dashboard, history, budget alerts, and insights at the same time. We used SQLite for financial records and their relationships, and SharedPreferences for lightweight settings such as dark mode.

We tried to keep responsibilities separate. The screens collect input and display information; state management coordinates updates; and the database layer stores and retrieves records. For example, saving a transaction begins with form validation, continues through the state and database code, and ends with the affected screens displaying the latest values. We found this separation especially helpful when we were developing different features at the same time.

4. Individual contribution – Akshitha

My main focus was the application's foundation and the parts that users interact with most often when tracking their day-to-day spending. I worked on the initial Flutter app structure, finance models, and core screens. Once the basic workflow was in place, I concentrated on making transaction entry and review more reliable.

Transaction validation and history improvements

I strengthened the transaction form validation and added tests for it. I then worked on searching transactions by description and category, filtering income and expenses separately, and sorting by date and amount. These changes were important because a transaction list becomes difficult to use when it grows. I wanted users to be able to find a specific purchase without scrolling through every entry.

Budget alerts and dashboard feedback

I implemented spending thresholds and the dashboard messages that appear when a user approaches or exceeds the budget. I also added tests for the threshold boundaries and widget tests for the dashboard's progress and alert display. One detail I paid attention to was the difference between being just below a threshold and being exactly at it; those small conditions matter for the message the user sees.

CSV export, testing, and documentation

I built the transaction CSV generator and added formatting and security-related tests. I then connected the export to native sharing through the Settings area so it could actually be used from the app. I also contributed the Android navigation integration smoke test and the held-out spending-helper evaluation work. Toward the end, I prepared AI_FEATURE.md and AI_Usage_Log.md, added final test-coverage evidence, and made smaller code-quality fixes. The supplied GitHub history includes my work across PRs #1, #2, #4, #7, #8, #9, #10, and #14.

5. Individual contribution – Mouni

My main focus was the financial insights and savings-challenge side of PennyPilot. I wanted these features to use the app's real financial data rather than display static sample results. That meant working not only on the screens but also on the services, shared state, and local database methods behind them.

Financial insights and analysis

I added the financial intelligence service foundation and connected it to the Provider-based state. I worked on the Insights dashboard and navigation, then improved time-aware analysis and its regression tests. I also added historical forecast accuracy with SQLite persistence and explainable unusual-spending detection. These features required careful handling of the data that was actually available, including cases where there was not enough history to show a meaningful result.

Savings challenges and contribution history

I worked on challenge validation, editing, filtering, and sorting. I then added a progress-history model, SQLite migration and database methods, and the FinanceState integration needed to connect contributions to progress. I worked on making contribution updates atomic so that a contribution and its progress update stay coordinated. I also implemented the contribution-history interface, which lets users see how their savings changed over time.

Export, theme, tests, and documentation

I developed savings-challenge CSV export and its tests and connected the feature to the savings workflow. I added transaction-screen widget tests and improvements that made state easier to test. I also implemented persistent dark mode and helped finalize screenshots, the README, architecture documentation, and test-coverage reporting. The supplied GitHub history includes PRs #3, #5, #6, #11, #12, and #13 related to my work.

6. What we worked on together

Even though we divided the main features, we could not build them as completely separate apps. We both had to make sure that our changes worked with the same finance models, SQLite database, shared state, and navigation. For example, Akshitha's transaction updates affect the financial information that Mouni's insights features analyze. Likewise, savings updates have to remain consistent with the rest of the application's state.

We discussed the user workflows and feature priorities together, integrated changes through GitHub, and checked that new functionality did not break existing screens. Both of us contributed to testing, debugging, documentation, and the final demonstration. We also prepared the PowerPoint and recorded the walkthrough together, with each of us explaining the features and implementation areas we worked on.

We used ClickUp to organize tasks under To Do, In Progress, Review, and Complete, and GitHub branches and pull requests to review and merge work. The final ClickUp evidence shows task assignments for shared items such as five-screen navigation, SQLite CRUD, Provider state management, savings persistence, and the offline helper. We do not interpret a shared task assignment as proof that both people wrote the same amount of code; the commit and PR history gives a more specific picture of individual contributions.

7. Challenges we faced and how we handled them

Keeping related screens up to date

One issue we had to think through was that financial information appears in several places. If an expense is edited or deleted, the History screen, dashboard totals, budget warning, and insights should not disagree. Using shared state and keeping database operations outside the widgets made those changes easier to manage and test.

Managing related savings data

Savings contributions introduced another challenge. Adding a contribution should not leave the goal's progress in a different state from the stored history. We addressed this by adding persistence for progress history and coordinating contribution operations with the database and state layer. We also spent time on validation and the behavior of linked records.

Handling missing and unusual data

We learned that an insights feature should not show a confident answer when the underlying data is incomplete. We added explanatory empty states for forecast accuracy and kept the Spending Check rule-based and transparent. We also tested budget thresholds and edge cases instead of only the usual successful flow.

Team integration and evidence

As we merged our work, we had to check for changes affecting shared files and test behavior. GitHub reviews and regression tests helped with this. We also realized that project-management evidence is much easier to collect while the work is happening. Our final ClickUp screenshots were captured on October 31, so we are not presenting them as screenshots from earlier dates. In a future project, we would save dated board snapshots and task-to-code links regularly.

8. Our offline Spending Check

We implemented a local rule-based Spending Check rather than a trained AI model. The helper compares spending with the selected budget: below 80% it gives an on-track message, from 80% to below 100% it warns that spending is approaching the limit, and at or above 100% it warns that the budget has been reached or exceeded. The helper also needs to account for missing or invalid budget data. Because the logic runs locally, the feature does not need an external AI API.

For the project evaluation, we documented the rules, a fixed-message baseline, test cases, and limitations in AI_FEATURE.md. We also considered counterexamples. Someone may have spent a large amount because of a necessary purchase, so the app's message should be treated as advice rather than an automatic judgment. The user remains in control of the budget and financial decisions.

9. Testing and final checks

We did not rely only on launching the app and clicking through the screens. Our tests covered transaction validation, history search/filter/sort, budget-alert boundaries, CSV generation, savings workflows, and UI behavior. We also added navigation smoke-test work and used the Android emulator to demonstrate the connected screens.

The final verification recorded 153 passing automated tests, zero Flutter analysis issues, and 80.28% line coverage (1,026 of 1,278 measured lines). We also built the release APK. These results helped us catch regressions, although we know coverage alone does not guarantee that every possible device or lifecycle issue has been tested. Our repository includes the tests and supporting evidence.

10. What we would improve next

If we had more development time, we would expand accessibility and testing on different screen sizes and devices, make the insights more helpful when there is only a small amount of history, and improve the context behind Spending Check recommendations. We would also refine the savings experience and continue strengthening database-related regression tests. On the teamwork side, we would document ClickUp checkpoints earlier and make the connection between a task, pull request, and test result easier to follow.

11. Conclusion

PennyPilot started with a simple idea: make it easier for students to see where their money goes. As we built it, we learned that a useful app needs much more than attractive screens. The data has to be stored reliably, the same values have to appear consistently across features, and changes need to be tested. Working on different parts of one connected application also gave us experience with dividing responsibilities and integrating code as a team.

We are pleased with the working offline app we were able to put together. More importantly, this project helped both of us understand Flutter, state management, local databases, testing, and team development in a much more practical way.

Links and supporting files

GitHub: https://github.com/MouniSri039/PennyPilot

ClickUp: https://app.clickup.com/90141780871/v/b/2kydwuw7-354

Supporting documents: README.md, ARCHITECTURE.md, AI_FEATURE.md, AI_Usage_Log.md, Kanban_Contribution_Evidence.pdf, and the team contribution report.

Appendix – Screenshots from our application

The screenshots below are from our PennyPilot repository. The recorded presentation shows the live workflows and code walkthrough.

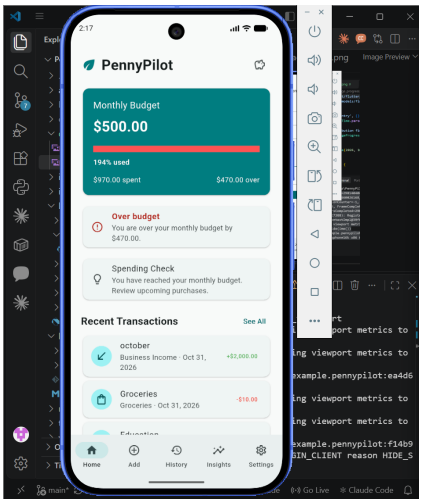


Figure 1. Home dashboard

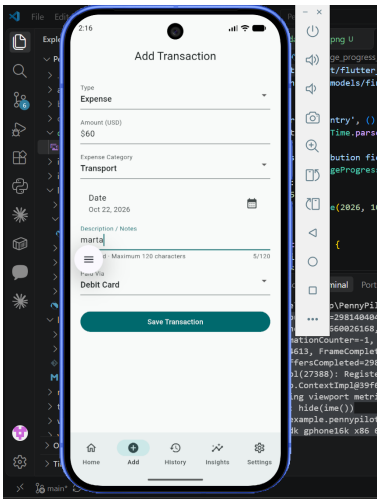


Figure 2. Adding a transaction

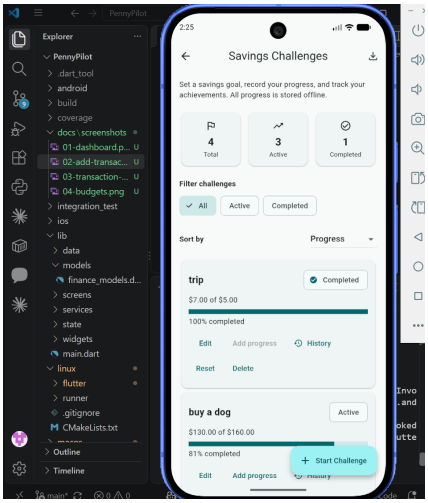


Figure 3. Savings challenges

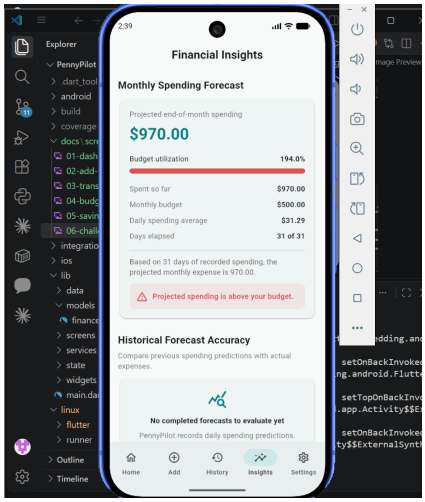


Figure 4. Financial insights